

Problem-solving assignment (10%) for 3325ENG Numerical Engineering Methods

Dr Stephen So

Trimester 2, 2026

1 Introduction

In this assignment, you will be evaluating several machine learning algorithms for the application of face recognition. Face recognition technology has becoming increasingly ubiquitous, since it is an unobtrusive and non-invasive biometric method, i.e. for verifying a person's identity or identifying them from a crowd of people.

The idea of face recognition is for a person's face to be captured by a camera and a machine (PC, raspberry pi, microcontroller, etc.) then compares the face to its stored database of "allowed" faces. Depending on the application and use, we have two different scenarios, which will be explored in this assignment:

- **Biometric verification:** This is the case where we wish to **verify** that the person is who he/she is claiming to be. In essence, we are using the face of the person as a password for secure access, e.g. checking the face of the 'client' at an ATM to verify their identity. The idea is for the system to reject other people's faces (the 'imposters') if they look different to the client. Therefore, the output of the system is a binary outcome: accept (1) or reject (0).
- **Biometric identification/recognition:** This is the situation where we wish to **identify** the person using their face. For example, we may wish to find a particular person from a crowd of people at a train station. In this case, the output of the system is a multi-class outcome, rather than binary: person 1, person 2, person 3, etc.

In Part 1 of this assignment, you will be developing a simple biometric identification system using the provided face database. You will be using PCA (principal component analysis) to perform dimensionality reduction before applying two different classifiers (logistic regression and support vector machines).

In Part 2 of this assignment, you will be developing a real-time biometric verification system using real-time face capture using a webcam on the PC. The scenario is a biometric system for an ATM, where the client is trying to access their money and the system is using their face to verify their identity.

Note that this assignment is focused on **traditional "shallow" machine learning** methods, rather than modern deep learning, transfer learning, etc.

You are to work in pairs for this assignment and you will need to distribute the workload amongst each member. You will be using a Jupyter notebooks for Part 1 and normal Python programs for Part 2 of the assignment.

1.1 Submission instructions

The **due date** for this assignment is Friday, 9 October 2026 at 11.59 pm. You are to submit your Jupyter notebook (*.ipynb) for Part 1 and Python scripts (*.py) for Part 2 as well as

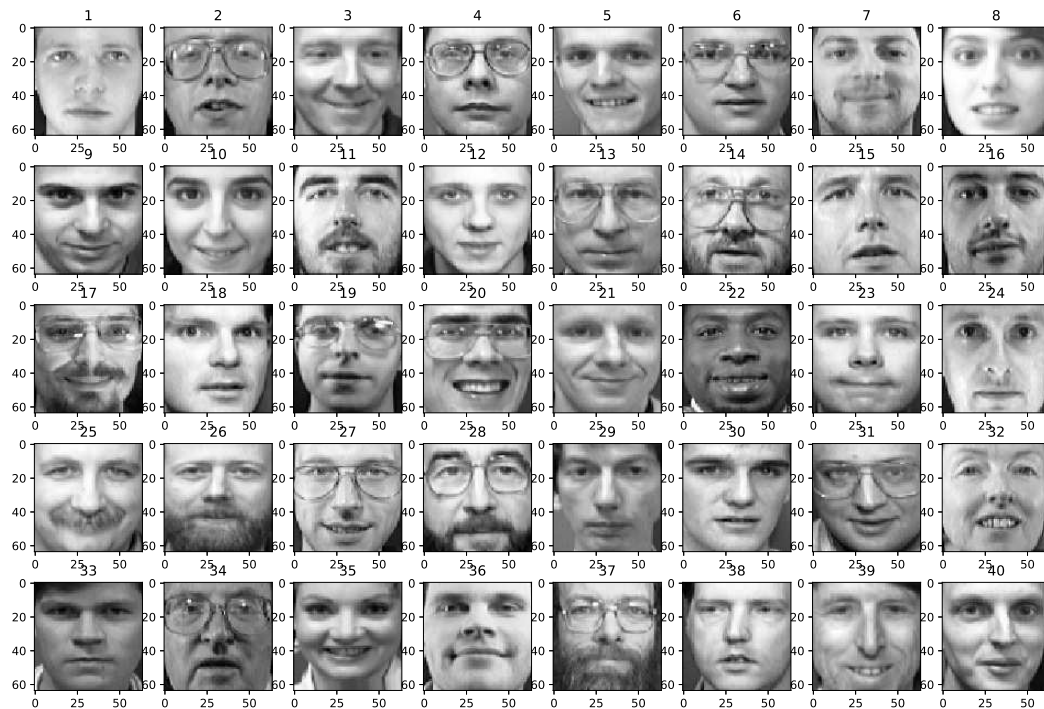


Figure 1: All 40 subjects from the face database

any other facial image files that are needed to run your program to the submission point on Learning@Griffith. You will also need to enrol in an assignment group with the other pair (if you have one) in order to submit the files.

You will also need to prepare a 12 minute video presentation of your assignment that demonstrates your final face recognition systems to potential clients working on ATM security. Both members of your group should be involved and a powerpoint slides presentation should be used to explain the specifics of your system. More details on how to prepare the video as well as what to mention will be detailed in a later section of this assignment.

The total number of marks in this assessment is 20 and the weighting of the assessment item is 10%. The breakdown of the marks is as follows:

- Part 1 – 7 marks
- Part 2 – 8 marks (4 marks for base system, 4 marks for extended system)
- Video demonstration – 5 marks

2 Part 1: Recognition from a face database (7 marks)

2.1 The face database

The face database that you will be using is stored in a file called `facedb.hdf5`, which is in HDF5 format¹. This face database consists of 40 people (referred to as ‘subjects’), as shown in Figure 1. Each person has 10 greyscale images (of size 64×64 pixels) of their faces, often in different facial orientations for a total of 400 images. An example of these 10 images is shown in Figure 2.

¹This is known as Hierarchical Data Format, which is used in industry and research organisation for storing huge amounts of data. For more information on how to access the data from HDF5, refer to the template Jupyter notebook on Learning@Griffith.

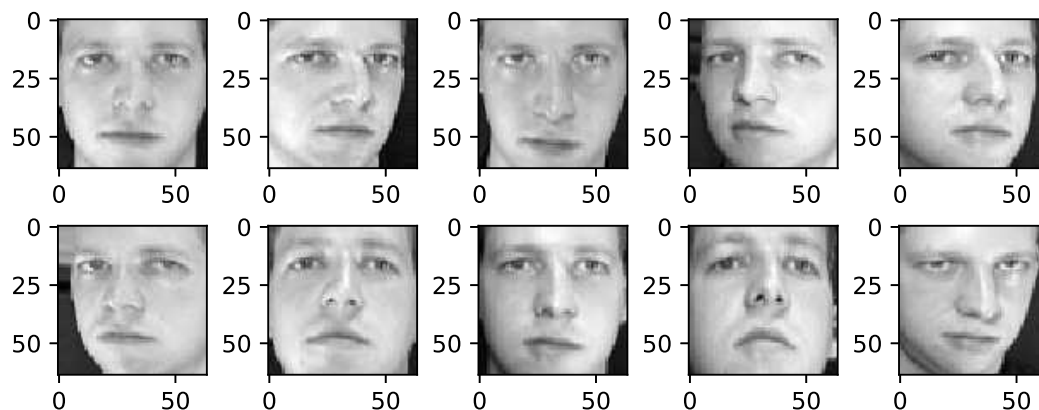


Figure 2: Ten different facial orientations of subject 1

Each face is stored in the HDF5 file as an $2D$ array of 64×64 floating-point values, where each value represents the luminance of the pixel. Associated with each face is a *label* (or target), which contains the **true identity** of the person. These labels, which consist of the numbers 1 to 40, are required for training our supervised learning classifier.

The database is split into **three sections**: a **training set**, a **validation set**, and a **test set**. The training set consists of a random selection of five faces for each subject, while the validation set has two faces and test set has three faces. There are a total of 200 faces in the training set, 80 faces in the validation set, and 120 faces in the test set.

2.2 The classification system

There are two phases to the classification system, as shown in Figure 3. In the **training phase**, the input image is a 64×64 array of pixel values and is passed to the *feature extraction* stage, which derives a *feature vector* that best represents the input and is useful for classification. In our case, we use the raw pixels from each face as a feature vector (there are other feature vector representations that can be used).

Each feature vector (of dimension 4096) is then passed to the feature transformation stage, which performs the process of *dimensionality reduction*, reduces the size of the vectors to K , so that it is suitable for the classification. This is especially important since the dimensionality of the feature vectors (4096) is significantly larger than the number of training examples (280) that we have, which is not ideal and can result in *over-fitting*².

The classifier then tries to learn the correct output when given the feature vectors and uses the provided correct labels as a guide. The labels, also known as *ground truths*, are provided to help the classification algorithm in correcting its learning. This is a feature of *supervised learning*. All feature vectors from the training set are passed to the classifier, where its training algorithm will find the best model weights that minimise classification error.

In the **testing phase**, each test image is processed in exactly the same way as in training (feature extraction and transformation) and the classifier uses its trained model to make a prediction. In our case, the classifier outputs the predicted subject when given a face.

2.3 Assignment tasks

In Part 1 of this assignment, you will be writing the face recognition system in a Jupyter notebook. You should perform the following tasks in this notebook:

²Recall that overfitting refers to a classifier that is so tightly tuned to the particular training set, it performs poorly on data that is not part of the training set, i.e. it does not generalise well.

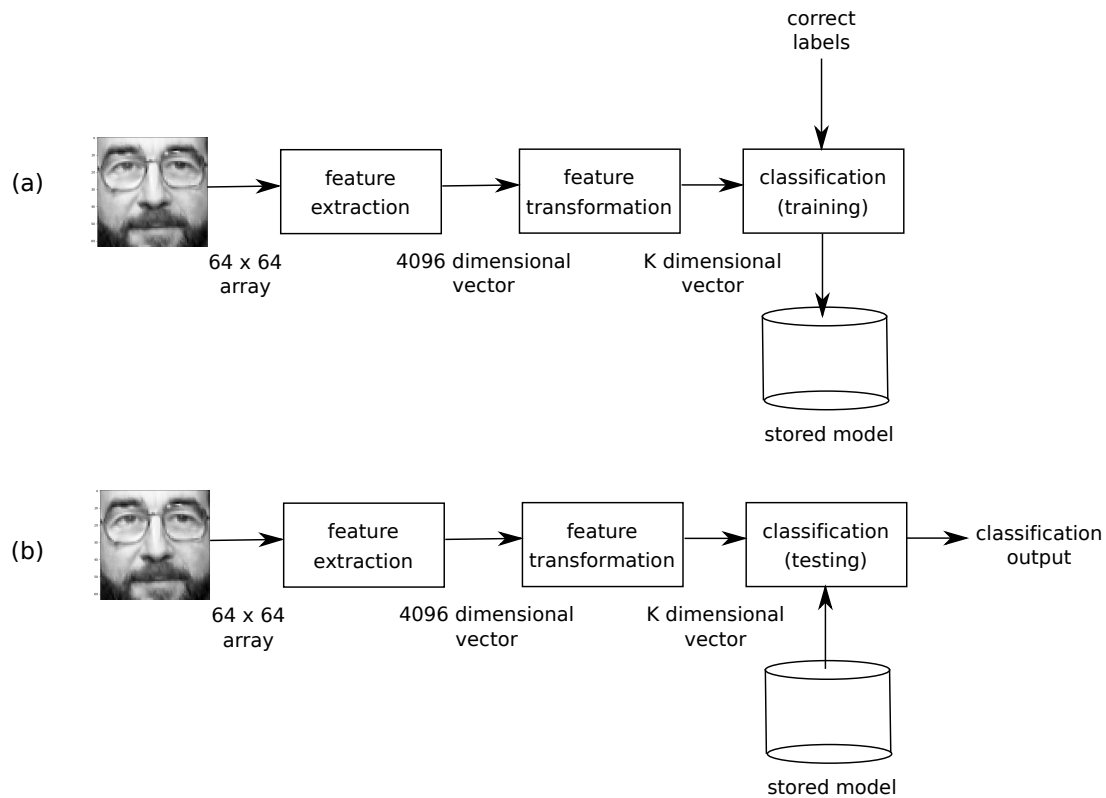


Figure 3: Block diagram of the face recognition system (biometric identification): (a) training phase; and (b) testing phase

1. Load the training, validation and test sets of the face database from the provided HDF5 file and display the first six faces from each set in separate figures. Use a subplot of 2×3 for each set.
2. Write your own Python code for performing PCA (principal component analysis) on the training set. For this, you will need to write your own code for computing the covariance matrix, then use the built-in routine `np.linalg.eig()`, and sort the eigenvectors according to their eigenvalues in descending order, resulting in the PCA transformation matrix $\mathbf{V}$.
3. Plot the sorted eigenvalues on a graph and determine a suitable number K of principal components to keep. Note that this number should be less than the number of training vectors, which is equivalent to the rank of the covariance matrix. Plot the common logarithm of the eigenvalues.
4. Perform PCA transformation on the zero-mean training set while keeping the first K principal components.
5. Write your own implementation of the linear discriminant analysis (LDA) classifier, using only the equations provided in the lecture slides. You should only use `numpy` routines for this. This should include a function for training the LDA classifier, return parameters such as the common covariance and mean, as well as a function for inference, where a feature vector is input and a class label is returned. Then train your LDA classifier on the face training database.
6. Repeat the previous step but now implementing the quadratic discriminant analysis (QDA) classifier instead, using only the equations provided in the lecture slides and using only `numpy` routines for this.

7. Use the logistic regression (LR) classifier to learn from the training set (using the provided training set labels). You can use the routine from *scikit-learn* for performing logistic regression. Do a grid search for an appropriate L2 regularisation term λ that gives the highest accuracy on the validation set. You should also plot the accuracy for both the training and validation sets for grid search values of λ . Note that in the scikit-learn routine, the regularisation term is $C = 1/\lambda$.
8. Use the support vector machine (SVM) classifier from the *scikit-learn* library. You should pick the optimal value for the regularisation term C that gives the highest accuracy on the validation set. You should plot the accuracy of the SVM classifier on both the training and validation sets for different values of C .
9. After picking the optimal parameters using the validation set, classify the faces from the test set and print the accuracy for each classifier (LDA, QDA, LR, SVM).
10. Determine which faces from the test set each classifier got wrong and use `imshow()` to print the wrong and correct faces next to each other using subplots. Can you determine from the faces why the classifier got it wrong?

3 Part 2: Real-time verification using facial data (8 marks – 4 marks for base system + 4 marks for extended system)

3.1 The face capture program

In Part 2 of this assignment, you will be writing a real-time face verification system. The scenario is you are developing the biometric verification system for an ATM, in which a camera in the ATM captures the face of the client which then verifies their identity. The output of this system is a binary decision: accept (1) or reject (0).

In order to capture the face in real-time, a Python program has been provided to you called `faceCapture.py`, which you can download from Learning@Griffith. The OpenCV library is used to access the webcam on your PC or laptop and displays it in a window. Note that in order to align the face, two circles are shown in the window where the client needs to align both their eyes. A rectangle is displayed which shows you where the image will be cropped. The saved image will be resized to 100×100 . Pressing the 's' key saves the face image into the current directory. You can press the 's' key multiple times to capture several images, which will be named as a prefix followed by a number that increments. For example, if the prefix is `steve`, then multiple presses of the 's' will generate filenames `steve0.png`, `steve1.png`, `steve2.png`, etc.

You are to use the `faceCapture.py` program to capture multiple face images of the **client** and **imposter** for training your classifier. This is why it is a good idea to have two people undertake this assignment so that you will have two faces to work with. It is recommended that each face is slightly different, so that these variations can be learnt by the classifier. Variations such as neutral, smiling, and frowning expressions, glasses on and off, and different illuminations of the face (by using a torch from your phone to illuminate the left and right side of the face) is recommended to account for the variations that occur at the ATM. These variations are essential to ensure your classifier is robust to variations.

3.2 Assignment tasks

1. Using the `faceCapture.py` program provided, capture multiple facial images (say a minimum of 30 for each) of the client and imposter (determine who is who in the group),

- ensuring that eye-based alignment is maintained. Try to introduce variations such as different facial expressions, glasses on-and-off, and lighting variations that you would expect at a real ATM.
2. Divide the 30 images of both client and imposter into a training set and validation set. A good allocation would be 25 images for training and five images for validation. Hence in total, there are 50 images in the training set (25 + 25) and 10 images in the validation set (5 + 5)
 3. Write a training program in Python `faceTrain.py`. Use the the OpenCV function `cv2.imread("filename.png", 0)` to open the PNG images and return a numpy array.
 4. For the **base system**, train an LDA classifier and a logistic regression (LR) classifier from the *scikit-learn* library on the raw features from the training set (do not use PCA). Choose an appropriate L2 regularisation term λ for LR that maximises the accuracy on the validation set. Save both trained models in the same file `faceModel.model` using the *joblib* routines `dump`³.
 5. Adapt the `faceCapture.py` program to include a test phase in another Python program called `faceTest.py`, where pressing the 'c' key will capture the face into an array. Load the trained LDA and LR models from your `faceModel.model` file using the `load` routine from *joblib*. Using the LR classifier, predict and print the output (accept and reject) as well as the confidence probability. Also do the same for the LDA classifier.
 6. Note that verification in a security context means that we should be more conservative in accepting, i.e. it is safer to reject if the classifier is not confident on the result. Using the probability output of the classifier, work out a way of making it more conservative.
 7. In the **extended system**, you are free to use any "shallow" classifier and custom feature extraction to **improve upon** the base system. Do some research on these. You will need to technically justify all design decisions and explain why you believe it works better. You are free to use Python libraries for this task, such as *scikit-learn*.

4 Video demonstration of the Part 2 system (5 marks)

As well as submitting your Jupyter notebook and Python programs (`faceTrain.py` and `faceTest.py`) for Parts 1 and 2, you will need to present your working system from Part 2 in a short 12 minute video.

This is the scenario for this presentation: you and your partner are developers that wish to sell your product to a security firm that deals with ATM security. You are to show your base system (which is cheaper) and the extended system (which requires a higher cost subscription). In this video, which should be presented by *both members of the group*, you are to:

1. Briefly describe your base real-time face verification system using a few powerpoint slides, its components, as well as how it works;
2. Demonstrate the face training phase via desktop screen capture of client and imposter, ensuring that you provide enough variations that would be expected in a real ATM, such as facial expressions, illuminations, etc.;
3. Demonstrate the operation of the testing phase via desktop screen capture using client and imposter as well as different variations (expressions, lighting, etc.);

³See https://scikit-learn.org/stable/modules/model_persistence.html for more details.

4. Try testing for **unknown imposters** (ones that have not been trained in the classifier); and
5. Demonstrate your extended system with client, imposter (and unknown if available) to show that it is more accurate.

It is recommended that you use a webcam from a PC (most University PCs will have a webcam) or laptop to capture each member of the group presenting their slides using the OBS Studio software, which you can freely download from the Internet⁴. OBS Studio can also be configured to perform desktop screen capture as well. A video on how to use OBS Studio will be included on Learning@Griffith.

You can use video editors, such as Shotcut⁵, to edit your video and stitch together the footage. Try to keep your video to about 12 minutes in length. Export it using x264 into an mp4 file and upload it to the Learning@Griffith assignment submission point using the Studio option.

You will be assessed on the following criteria:

- Presentation skills (clarity of explanation, eye contact, intonation, etc.)
- Technical rigour (accuracy and validity of the explanation, correct use of technical terms, justification of design decisions, etc.)
- System demonstration (meets technical requirements, accuracy of the system, robustness to variations demonstrated, robustness to ‘unknown imposters’)

⁴You can download OBS studio from <https://obsproject.com/download>

⁵Shotcut is free software that you can download from <https://shotcut.com/downloads/>